SAGE at AraGenre 2026: Adapter-Refined Definition Embeddings for Zero-Shot Arabic Genre Classification

Yasmina Mansour[1,2,*], **Ahmad Afles**[1,*], **Mohamad Ballout**[2]
[1]Arab Center for Research and Policy Studies (ACRPS)
[2] American University of Beirut (AUB)
Correspondence: ymansour@dohainstitute.edu.qa

Abstract

We present our system SAGE: **S**entence **A**nalysis **G**enre **E**valuation for the AraGenre shared task, which assigns Arabic sentences a hierarchical genre label from an English definition alone, with none of the 74 evaluation genres represented in the official training data. Since a definition is not itself an instance of its genre, we fine-tune the encoder generating the embeddings of both the sentences to classify and the candidate genre definitions and train a small adapter that refines those of the definitions. Studying three encoders and four datasets that were generated for training purposes, we find that the adapter's training data matters more than either encoder choice or adapter hidden layer. Our best system, combining a fine-tuned encoder and an adapter, reached a Hierarchical Macro F1 score of 0.5278, placing seventh.

1 Introduction

A text's genre captures its communicative purpose, such as instructing, announcing or persuading rather than the subject matter it discusses. Arabic text classification has largely used flat label sets, whereas hierarchical label classification received little attention (Hamzaoui et al., 2025). The AraGenre shared task (El-Haj et al., 2026) addresses that gap with a two-level Arabic taxonomy of 6 broad genres and 74 specific genres (Figure A1).

The task poses several challenges. First, its official training data covers only 7 specific genres, none of which reappear among the 74 in the test set, making the task zero-shot in that no human-labeled example of any evaluation genre is available for training. Second, only a label and one-sentence definition are provided for each genre in English, from which the system must decide whether a given Arabic sentence belongs to it.

Our system classifies a sentence by comparing its embedding to that of each candidate specific genre's definition, assigning the label whose vector is closest, and deriving the broad genre from the taxonomy. However, a definition describes a genre without necessarily being an instance of it, so its embedding may sit far from those of the texts it is meant to match. To narrow this gap, our approach either fine-tunes the encoder that produces these embeddings, trains a small adapter that refines the definition's embedding alone, or does both. Since the officially provided training data was too limited to support this, we generated additional datasets.

This paper presents our system for the AraGenre shared task, where our best-performing configuration pairs a fine-tuned encoder with a trained adapter, reaching a Hierarchical Macro F1 score of 0.5278, and placing seventh overall. Beyond this result, we contribute a study across three encoders and four generated datasets, examining where in the pipeline adaptation is most usefully applied. Our best-performing system's code is available here.

2 Background

2.1 Task Overview

The system is given the sentences to classify alongside the candidate genres and their English definitions, and is supposed to output a predicted broad and specific genre for each sentence. For instance, given the sentence:

كيف أستعيد ثقتي بنفسي بعد تجربة فاشلة؟

The expected output is the broad genre *Interactive* and the specific genre *Advice Columns*.

All example sentences are in Arabic, while genres and their definitions are in English; Table 7 summarizes sentence counts and genre coverage across the official splits and the datasets we additionally generated for training.

*These authors contributed equally to this work

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 288–294
October 28-29, 2026 ©2026 Association for Computational Linguistics

2.2 Related Work

Arabic text classification research has concentrated on flat category schemes, spanning topic classification (Elnagar et al., 2020; Abbas et al., 2011) and genre classification (Al-Yahya, 2019; Ferhat et al., 2024), with hierarchical classification such as the work of Aljedani et al. (2021) receiving comparatively little attention (Hamzaoui et al., 2025). Another example is Bouchiha et al. (2025), who fine-tuned AraGPT2 (Antoun et al., 2021) on a two-level Arabic taxonomy built from Wikipedia (WiHArD) (Bouchiha et al., 2024), spanning only 3 broad categories and 9 specific categories. Their training and test data are drawn from the same source, so every test category also appears in training. AraGenre differs on both counts: its taxonomy is substantially larger, covering 6 broad genres and 74 specific genres, and its train and test sets are constructed independently, so most specific test genres have no labeled examples in training at all.

Genre and register classification have been studied extensively in other languages, such as through the CORE corpus (Laippala et al., 2023), an English web-text taxonomy of 8 broad registers and 47 specific subregisters. Henriksson et al. (2024) extended this taxonomy to 16 languages, finding that performance dropped by an average of 7% on unseen languages, though the taxonomy itself stayed fixed and known throughout. AraGenre poses the reverse challenge: the language is known, but the specific genres themselves are unseen at training time.

3 System Overview

3.1 Dataset Generation

The officially released training data covers relatively few specific genres (Table 7). To help our systems generalize beyond this narrow set, we defined a broader set of genre categories, drawing in part on existing register and genre taxonomies for other languages (Laippala et al., 2023; Henriksson et al., 2024). Four datasets were constructed (Table 7), using the same three-stage pipeline described in Appendix C. G-D2, G-D3, and G-D4 were built from these additional categories before the official test set was released. G-D1, by contrast, was built afterward and exclusively from the officially released test-set taxonomy: its examples were generated solely from the published genre names and definitions. No labeled test instance was used at any stage of G-D1's construction. All these datasets can be found here.

3.2 Dataset Usage

The official dev split is used throughout this section to report the different systems' performance, while the official test split is reserved for the final evaluation of submitted systems. During training, each of the generated datasets was split per genre, with 15% of each genre's examples reserved for validation. This split is used both for validation-based early stopping and to ensure a rigorous evaluation process; validation examples are never used for training.

3.3 Evaluation Metrics

Systems are evaluated separately at the broad and specific genre levels. The official ranking metric is Hierarchical Macro F1 (Appendix D).

3.4 Vanilla Encoder

At inference time, the specific-genre definitions and the input sentence to be classified (t) are passed to an encoder, which returns their corresponding embeddings; each definition's embedding serves directly as that genre's prototype $\mathbf{p}_g$, i.e. $\mathbf{p}_g = f(\text{definition}_g)$, where f is the encoder used for embedding generation. The embedding of t is then compared to every $\mathbf{p}_g$ using cosine similarity, and the text is assigned the label of the specific genre with the highest score ($\hat{s}$):

$$\hat{s} = \arg\max_{g \in \mathcal{G}_s} \cos\big(f(t), \mathbf{p}_g\big) \tag{1}$$

Since every specific genre belongs to exactly one broad genre, the broad genre is never predicted on its own: specific genre $\hat{s}$ is predicted then the broad genre $\hat{b}$ is simply looked up from the taxonomy.

We evaluated three encoders: NeoAraBERT (Abou Chakra et al., 2026), chosen for its diacritics-aware tokeniser, its morphological segmentation, and its pre-training mixture spanning Modern Standard, Classical, and dialectal Arabic, using mean pooling; and Qwen3-Embedding (Zhang et al., 2025) at the 0.6B and 4B scales, using last-token pooling following its documented usage.

Results for all three encoders are reported in Table 1.

Exp.	Encoder	F1 score
1	Qwen3-Emb.-4B	0.557
2	NeoAraBERT	0.50
3	Qwen3-Emb.-0.6B	0.457

Table 1: Prediction F1 score using vanilla encoders.

3.5 Preprocessing

Since NeoAraBERT, a strictly Arabic language model, is used in most of our experiments, and the examples are given in Arabic the only preprocessing required is translating the genre definitions into Arabic.

3.6 Encoder Fine-Tuning

The vanilla encoders evaluated in Section 3.4 were not trained to capture genre similarity: their embeddings reflect general-purpose semantic representations rather than the specific relationship between a genre definition and an example text belonging to it. To make the encoders better capture this relationship, the encoders were fine-tuned using a contrastive learning approach (Focal-Contrastive and Contrastive Loss paragraphs, Appendix E) using only the example sentences in the datasets. Thus the encoder's embedding space is reshaped so that texts of the same genre are drawn together before any genre-definition embedding is computed. Encoders were fine-tuned in one of two ways: full-model fine-tuning, or LoRA (Hu et al., 2022) adapters (r=16, α=32) applied to all attention projection matrices of a frozen base model. Table 2 details the hyperparameters for both regimes.

Hyperparameter	Full-model Finetune	LoRA
Temperature T	0.05	0.1
Batch size	16	8
Optimizer	AdamW	AdamW
Learning rate	2×10^{-5}	2×10^{-4}
Max epochs	15	15
Patience	3	5

Table 2: Hyperparameters for the two encoder training regimes.

The fine-tuned encoders were later used for hierarchical genre prediction for an input sentence t, in the same way discussed in Sec. 3.4. Results of this fine-tuning procedure, across different encoders and configurations are reported in Table 3.

Exp.	Encoder	Loss	Dataset	F1 score
1	NeoAraBERT	Contrastive	G-D2	0.550
2	Qwen3-0.6B	Focal-Cont. (γ=2.0)	G-D2	0.557
3	Qwen3-0.6B (LoRA)	Focal-Cont. (γ=2.0)	G-D2	0.72
4	Qwen3-0.6B (LoRA)	Focal-Cont. (γ=2.0)	G-D4	0.665
5	NeoAraBERT	Contrastive	G-D2, G-D1	0.836

Table 3: Prediction F1 score using fine-tuned encoders. Exp. 1, 2, and 5 use full-model fine-tuning; Exp. 3 and 4 use LoRA (Table 2).

3.7 Adapter Training

Even a well fine-tuned encoder does not resolve this problem: a definition remains a description of the genre rather than an instance of it, so its embedding can still sit apart from the embeddings of actual genre examples, and the similarity in Equation 1 is computed across that gap.

As a solution, we apply a small residual adapter (A) to the definition's embedding ($\mathbf{d}$):

$$A(\mathbf{d}) = \text{normalize}\big(\mathbf{d} + W_2\,\text{ReLU}(W_1\mathbf{d})\big), \quad (2)$$

where $W_1 \in \mathbb{R}^{h \times d_{\text{model}}}$ and $W_2 \in \mathbb{R}^{d_{\text{model}} \times h}$ are learned weight matrices with hidden dimension h, where d_{model} is the dimension of the encoder's output embedding.

First, each training text and each genre's definition are embedded once. Then, at each epoch only the definition embeddings are passed to the adapter to form prototypes $\mathbf{p}_g$ while the training-text's remain fixed. The adapter's weights are updated by comparing the two using one of the following loss functions defined in Appendix E: Flat, Focal or Hierarchical, depending on the system configuration, as shown in Table 4.

Exp.	Encoder	Hidden	Loss	Dataset
1	NeoAraBERT**	128	Focal (γ=2.0)	G-D1
2	NeoAraBERT**	64	Focal (γ=2.0)	G-D1
3	Qwen3-4B	128	Focal (γ=2.0)	G-D4
4	NeoAraBERT**	64	Focal (γ=2.0)	G-D4
5	Qwen3-0.6B	64	Flat	G-D4
6	Qwen3-4B	64	Flat	G-D3
7	NeoAraBERT**	64	(Eq. 4) (λ_1=0.8, λ_2=0.2)	G-D3
8	NeoAraBERT**	256	(Eq. 4) (λ_1=0.8, λ_2=0.2)	G-D3

Table 4: Adapter training configurations. NeoAraBERT** is the fine-tuned encoder in Exp. 1 of Table 3.

All adapter-training runs use a fixed temperature $T = 0.1$ and learning rate 10^{-3}; the remaining hyperparameters vary by experiment group, as summarised in Table 5.

Hyperparameter	Exp. 1–2	Exp. 3–6	Exp. 7–8
Max epochs	30	60	60
Patience	N/A	5	5
Optimizer	Adam	Adam	AdamW

Table 5: Adapter training hyperparameters by experiment group. Experiments 1–2 train for a fixed 30 epochs without a validation-based stopping criterion.

At inference, labels are assigned as in Section 3.4,

but each prototype $\mathbf{p}_g$ is now $A(f(\text{definition}_g))$ rather than $f(\text{definition}_g)$ directly.

3.8 External Tools and Libraries

All systems were implemented in PyTorch using Hugging Face transformers; full library versions are in Appendix F.

4 Results and Analysis

Table 6 reports the adapter-based systems on both splits, scored with Hierarchical Macro F1 as used for the official ranking. Several rows differ in a single component, so we discuss them by design decision rather than by rank.

Exp.	Encoder	Hidden	Dataset	dev	Test
1	NeoAraBERT**	128	G-D1	0.8400	**0.5278**
2	NeoAraBERT**	64	G-D1	0.8230	0.5103
3	Qwen3-4B	128	G-D4	0.8700	0.3604
4	NeoAraBERT**	64	G-D4	0.8180	0.3409
5	Qwen3-0.6B	64	G-D4	0.7930	0.3359
6	Qwen3-4B	64	G-D3	0.8580	0.3270
7	NeoAraBERT**	64	G-D3	0.7818	0.3486
8	NeoAraBERT**	256	G-D3	0.8636	0.3319

Table 6: Adapter training configurations along with their Hierarchical Macro F1 on the dev. and test splits. NeoAraBERT** is the fine-tuned encoder in Exp. 1 of Table 3.

Adapter Training Data. The adapter's training dataset is the factor that splits the scores in Table 6 most sharply. Experiments 1 and 2 use G-D1, whose genres are those of the test set, and reach 0.5278 and 0.5103 respectively. Experiments 3 to 8 use G-D3 or G-D4, whose categories were defined independently of the evaluation taxonomy, their scores fall between 0.3270 and 0.3604. Moreover, Experiments 2 and 4 differ mostly in whether the adapter saw examples of the evaluation genres, yet stand 0.17 points apart on test.

Bottleneck width. Experiments 1 and 2 are identical apart from the hidden dimension of the adapter in Equation 2. Widening it from 64 to 128 gains 0.0172 points on test and 0.017 on dev. The same comparison in Experiments 7 and 8, where the width goes from 64 to 256, moves the dev score up by 0.082 points but the test score down by 0.017, so the wider adapter fits the dev genres more closely without transferring that advantage.

Encoder choice. Under comparable adapter settings on G-D4, Qwen3-Embedding-4B reaches 0.3604, the fine-tuned NeoAraBERT 0.3409 and Qwen3-Embedding-0.6B 0.3359. This spread is narrower than the one produced by changing the adapter's training data, and narrower than the spread among the same three models before any adaptation in Table 1, which runs from 45.7 to 55.7.

Dev and test. Every system scores higher on the dev set than on the test set, and system rankings sometimes reverse between the two splits as Experiments 7 and 8 show. The two splits differ in more than size: dev sentences are short, unambiguous, and each clearly represents a single genre, while test sentences are scraped from the web and carry that process's noise. All submissions used the released test texts unmodified.

Conclusion

We described our system for the AraGenre shared task, which assigns Arabic sentences a hierarchical genre label from English definitions alone, with every evaluation genre unseen in the official training data. Our pipeline compares a sentence embedding against one vector per specific genre and derives the broad genre from the taxonomy. Because a definition is not itself an instance of its genre, we fine-tuned the encoder generating the embeddings and we trained a small adapter over the definition embeddings. And for training purposes, we generated example sentences from the published genre names and definitions. Our best configuration combined both a fine-tuned encoder and an adapter, and it reached 0.5278 Hierarchical Macro F1 score and was ranked seventh.

Across the systems we submitted, the dataset an adapter was trained on separated the results more than the choice of encoder or the capacity of the adapter, where adapters trained on the evaluation genres transferred markedly better. Dev scores also exceeded test scores throughout, and the two splits did not always rank systems in the same order, which limits how far tuning on the dev set can be trusted when the evaluation genres are unseen. An additional clear limitation of our approach is that the generated examples come from language models rather than from real Arabic writing.

Acknowledgments

We thank the AraGenre organisers for designing the shared task and for their support throughout the evaluation period.

References

Mourad Abbas, Kamel Smaïli, and Daoud Berkani. 2011. Evaluation of topic identification methods on Arabic corpora. *Journal of Digital Information Management*, 9(5):185–192.

Chadi Abou Chakra, Hadi Hamoud, Osama Rakan Al Mraikhat, Qusai Abu Obaida, Mohamad Ballout, and Fadi A. Zaraket. 2026. NeoAraBERT: A modern foundation model for Arabic embeddings with diacritics-aware tokenization and POS-targeted masking. Technical report, Arab Center for Research and Policy Studies.

Maha Al-Yahya. 2019. A comparative study of machine learning methods for genre identification of classical Arabic text. *Computers, Materials & Continua*, 60(2):421–433.

Nawal Aljedani, Reem Alotaibi, and Mounira Taileb. 2021. HMATC: Hierarchical multi-label Arabic text classification model using machine learning. *Egyptian Informatics Journal*, 22(3):225–237.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2021. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 196–207, Kyiv, Ukraine (Virtual).

Djelloul Bouchiha, Abdelghani Bouziane, Naouel Doumi, Faiza Berbouchi, Amina Kebir, Nada Mebarki, and Bilal Benameur. 2024. WiHArD: Wikipedia based hierarchical Arabic dataset for text classification. In *Proceedings of the 4th International Conference on Embedded & Distributed Systems (EDiS)*, pages 115–118.

Djelloul Bouchiha, Abdelghani Bouziane, Noureddine Doumi, and Benamar Hamzaoui. 2025. Fine-tuning AraGPT2 for hierarchical Arabic text classification. *Science, Engineering and Technology*, 5(1):55–65.

Mo El-Haj, Saad Ezzini, Shadi Abudalfa, Mustafa Jarrar, Nguyen Minh Chi, and Nguyen Minh Quan. 2026. Aragenre 2026: A hierarchical definition-guided arabic genre classification shared task. In *Proceedings of the 4th Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics.

Ashraf Elnagar, Ridhwan Al-Debsi, and Omar Einea. 2020. Arabic text classification using deep learning models. *Information Processing & Management*, 57(1):102121.

Zeyd Ferhat, Abir Betka, Riyadh Barka, Zineddine Kahhoul, Selma Boutiba, Mohamed Tiar, Habiba Dahmani, and Ahmed Abdelali. 2024. Functional text dimensions for Arabic text classification. In *Proceedings of the Second Arabic Natural Language Processing Conference*, pages 352–360, Bangkok, Thailand. Association for Computational Linguistics.

Google DeepMind. 2026. Gemini 3.5 Flash-Lite. https://deepmind.google/models/model-cards/gemini-3-5-flash-lite/. Model card, accessed 2026.

Benamar Hamzaoui, Djelloul Bouchiha, and Abdelghani Bouziane. 2025. A comprehensive survey on arabic text classification: progress, challenges, and techniques. *Brazilian Journal of Technology*, 8(1):e77611.

Erik Henriksson, Amanda Myntti, Anni Eskelinen, Selcen Erten-Johansson, Saara Hellström, and Veronika Laippala. 2024. Automatic register identification for the open web using multilingual deep learning. *Preprint*, arXiv:2406.19892.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Veronika Laippala, Samuel Rönnqvist, Miika Oinonen, Aki-Juhani Kyröläinen, Anna Salmela, Douglas Biber, Jesse Egbert, and Sampo Pyysalo. 2023. Register identification from the unrestricted open Web using the Corpus of Online Registers of English. *Language Resources and Evaluation*, 57(3):1045–1079.

OpenAI. 2024. GPT-4o system card. *Preprint*, arXiv:2410.21276.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, and Junyang Lin. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *Preprint*, arXiv:2506.05176. Author list truncated; full list in the original preprint.

A Genre Hierarchy

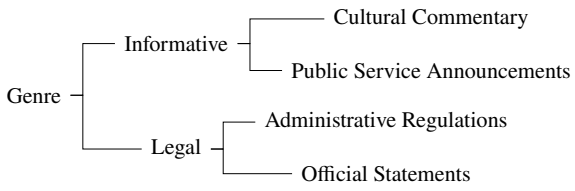


Figure A1: Excerpt from the AraGenre genre taxonomy, illustrating the broad–specific hierarchy.

B Dataset Statistics

Split	#Sentences	#Broad	#Specific
Train	140	5	7
dev	110	6	6
Test	27,972	6	74
G-D1	888	6	74
G-D2	930	–	31
G-D3	2,510	6	44
G-D4	2,860	-	86

Table 7: Statistics of the AraGenre training, dev, and test splits, together with the additional datasets generated for this work.

C Dataset Generation Pipeline

Each genre's examples were generated from its definition in an isolated session using Gemini 3.5 Flash-Lite (Google DeepMind, 2026), so examples from different genres would not influence each other. The generated texts were then checked for redundancy by GPT-4o (OpenAI, 2024), which flagged shared openings and repeated templates that would reduce within-genre variance. Finally, examples were reviewed manually; since real text often blends multiple genres, sentences representing more than one genre were kept rather than discarded.

D Evaluation Metrics

Hierarchical Macro F1. The official ranking metric combines performance at both levels of the taxonomy into a single score. Let $F1_{\text{broad}}$ denote the Macro F1 computed over the 6 broad genre labels, and $F1_{\text{specific}}$ denote the Macro F1 computed over the specific genre labels. The Hierarchical Macro F1 is their average:

$$F1_{\text{hier}} = \frac{F1_{\text{broad}} + F1_{\text{specific}}}{2}$$

Because $F1_{\text{broad}}$ is derived from the predicted specific genre via the taxonomy lookup (Section 3.4), the two terms are not independent.

E Loss Functions

Focal-Contrastive. Used during encoder fine-tuning. A batch contains B same-genre anchor-positive pairs $(\mathbf{A}_i, \mathbf{P}_i)$ of sentences where $\mathbf{P}_i$ is the correct match for $\mathbf{A}_i$ and every other positive sentence in the batch acts as a negative.

Let $\mathbf{a}_i$ be the embedding of $\mathbf{A}_i$ and $\mathbf{p}_i$ be that of $\mathbf{P}_i$. Then for each anchor i and every positive j in the batch, the following logit is calculated:

$$z_{i,j} = \frac{\cos(\mathbf{a}_i, \mathbf{p}_j)}{T}$$

These logits are converted into a probability distribution over the batch via softmax; $P(i)$ denotes the probability assigned to anchor i's own positive, $\mathbf{P}_i$:

$$P(i) = \frac{e^{z_{i,i}}}{\sum_{j=1}^{B} e^{z_{i,j}}}$$

The loss then penalizes the model based on this probability, encouraging it to be high:

$$\mathcal{L}_{\text{focal-contrastive}} = -\frac{1}{B}\sum_{i=1}^{B}\big(1 - P(i)\big)^{\gamma}\log P(i)$$

Contrastive Loss. The same in-batch anchor, positive setup as Focal-Contrastive above, B same-genre anchor–positive pairs, with logits $z_{i,j}$ and softmax probability $P(i)$ defined identically:

$$\mathcal{L}_{\text{contrastive}} = -\frac{1}{B}\sum_{i=1}^{B}\log P(i)$$

Flat. A standard softmax classification loss over the specific-genre space. For each genre g with embedding $\mathbf{p}_g$ and sentence embedding $\mathbf{v}$, the logit is:

$$z_g = \frac{\cos(\mathbf{v}, \mathbf{p}_g)}{T} \tag{3}$$

and the softmax probability assigned to the correct genre y is:

$$P(y) = \frac{e^{z_y}}{\sum_{g' \in \mathcal{G}_s} e^{z_{g'}}}$$

The loss penalizes low-confidence predictions:

$$\mathcal{L}_{\text{flat}} = -\log P(y)$$

Focal. The same classification setup as Flat, used during adapter training, reweighted so that already well-separated examples contribute less to the loss than confusable ones:

$$\mathcal{L}_{\text{focal}} = -\big(1 - P(y)\big)^{\gamma}\log P(y)$$

Hierarchical. Extends the Flat loss with a term for broad genres. A broad genre b's score pools the logits (Equation 3) of its specific genres C_b via log-mean-exp, which keeps broad genres with

more specific genres from scoring higher than others:

$$\mathcal{L}_{\text{broad}}(b) = \log \frac{1}{|C_b|} \sum_{g \in C_b} e^{z_g}$$

The final loss combines the specific-genre loss and the broad-genre term, each independently weighted:

$$\mathcal{L}_{\text{hier}} = \lambda_1 \mathcal{L}_{\text{flat}} + \lambda_2 \mathcal{L}_{\text{broad}} \quad (4)$$

F Libraries & Repositories

All systems were implemented in PyTorch[1] using Hugging Face transformers[2]. NeoAraBERT required transformers==4.49.0; Qwen3-Embedding required transformers>=4.51.0 via sentence-transformers[3] (>=2.7.0).

Our best achieving system's code can be found here. Trained models were stored on Hugging Face[4].

[1] https://pytorch.org/
[2] https://github.com/huggingface/transformers
[3] https://github.com/UKPLab/sentence-transformers
[4] Yasmina Mansour's account, Ahmad Afles' account